



Master 1 GIL - XML
Projet : sepahub

RAPPORT MINI PROJET

Version : 2.0

Date : 24/09/2026

Rédigé par : Florian Pépin

Note : *Fork et refonte du projet SEPA26-REST*

Table des matières

1	Introduction	2
2	Architecture du Projet	3
2.1	Organisation Logicielle	3
2.2	Stack Technique	3
2.3	Points d'accès	3
2.3.1	API REST	3
2.3.2	Interface Utilisateur	5
3	Patrons de Conception	6
3.1	Patron Builder (Monteur)	6
3.1.1	Motivation	6
3.1.2	Correspondance des Acteurs	6
3.2	Patron Abstract Factory (Fabrique Abstraite)	7
3.2.1	Motivation	7
3.2.2	Correspondance des Acteurs	7
3.3	Patron Chain of Responsibility (Chaîne de Responsabilité)	8
3.3.1	Motivation	8
3.3.2	Correspondance des Acteurs	8
4	Description de l'architecture Docker et Déploiement	9
4.1	Architecture Multi-Services	9
4.2	Sécurité et Isolation Réseau	9
4.3	Maintenance et Cycle de Vie	9
A	Inventaire des Entités par Paquetage	10

1 Introduction

Ce projet porte sur la réalisation du service REST SEPA26 et de son interface Web. L'objectif est de construire une architecture basée sur la norme ISO 20022 (prélèvement SEPA Direct Debit, format `pain.008.001.02`) fournie en cours pour gérer des transactions bancaires. Le déploiement est entièrement conteneurisé et orchestré via Docker Compose (Frontend Angular, Backend Spring Boot et Base de données MariaDB).

2 Architecture du Projet

L'application est structurée selon un modèle classique en couches (Layered Architecture), garantissant une stricte séparation des responsabilités conformément aux principes SOLID et DRY.

2.1 Organisation Logicielle

L'application suit une architecture en couches, dont voici les responsabilités principales :

- **Couche de Présentation (API REST)** : Gère les requêtes HTTP via des contrôleurs REST stricts (`@RestController`), assure le rendu en XML (JAXB) et JSON (Jackson), la transformation HTML via XSLT (Saxon-HE), et la normalisation des réponses d'erreur via un gestionnaire global (`@RestControllerAdvice`).
- **Couche de Service** : Centralise la logique métier SEPA, orchestre la validation technique et métier, et délègue les transformations XML à des services spécialisés.
- **Couche d'Accès aux Données** : Assure la persistance relationnelle via Spring Data JPA et MariaDB.
- **DTO & Utilitaires** : Facilitent le transfert de données, l'adaptation temporelle et les transformations XSLT.

2.2 Stack Technique

Le service repose sur une stack Java 25 moderne et robuste :

- **Framework** : Spring Boot 4 (Spring Web, Spring Data JPA).
- **Gestion des dépendances** : Maven.
- **Stockage** : MariaDB 11 (Production/Docker) et H2 (Environnement de test).
- **Traitement XML** : JAXB (Jakarta XML Binding), Jackson-XML et Saxon-HE (moteur XSLT 2.0/3.0 et XPath).
- **Interface Frontend** : Angular 22 (Application Single Page autonome servie par Nginx).

2.3 Points d'accès

L'application expose une architecture découplée composée d'une API REST backend et d'une interface web utilisateur (Single Page Application Angular).

2.3.1 API REST

L'ensemble des services métiers de manipulation et consultation des transactions SEPA sont exposés sous le préfixe `/sepa26`. Ces points d'accès sont également accessibles directement via le reverse-proxy Nginx configuré sur le frontend (port 4200) :

Méthode	Endpoint	Paramètres	Description / Formats
GET	/sepa26/resume/xml	Aucun	Liste des 10 derniers prélèvements (Retour <code>application/xml</code>)
GET	/sepa26/xml/{id}	id (chemin)	Document complet ISO 20022 conforme XSD (<code>application/xml</code>)
GET	/sepa26/html/{id}	id (chemin)	Fiche détaillée générée dynamiquement via transformation XSLT (<code>text/html</code>)
POST	/sepa26/insert	validate (opt.)	Insertion d'un prélèvement (<code>consumes: application/xml</code> , <code>produces: application/xml</code>). Évalué via la chaîne de responsabilité.
DELETE	/sepa26/delete/{id}	id (chemin)	Suppression définitive d'une transaction et statut <code>DELETED</code> (<code>application/xml</code>)
GET	/sepa26/search	date, sum (opt.)	Recherche multicritères par date minimale et montant minimal (<code>application/xml</code>)

TABLE 1 – Points d'accès de l'API REST Backend (/sepa26)

2.3.2 Interface Utilisateur

L'interface utilisateur de l'application est intégralement prise en charge par l'application moderne Angular (servie sur le port 4200), qui consomme les points d'accès de métadonnées et de documentation de l'API :

Composant	URL / Route	Fonctionnalité
Frontend Angular	/ ou /transactions	Tableau de bord et liste des 10 dernières transactions avec suppression modale
Frontend Angular	/transactions/:id	Consultation détaillée avec visualiseur XML indenté et copie presse-papier
Frontend Angular	/transfert	Dépôt et saisie de document XML avec validation XSD et notification toast
Frontend Angular	/search	Interface de recherche filtrée par date et montant
Frontend Angular	/help	Interface de documentation interactive consommant dynamiquement le catalogue JSON du backend
Backend REST	http://localhost:8100/	Métadonnées de l'application et statut de santé en JSON (application/json)
Backend REST	/help ou /sepa26/help	Catalogue exhaustif des routes au format JSON consommé par le frontend

TABLE 2 – Routes de l'Interface Web et Documentation

3 Patrons de Conception

Pour améliorer la lisibilité, la modularité et la maintenabilité du backend, trois patrons de conception du Gang of Four (GoF) ont été mis en place : le **Builder**, l'**Abstract Factory** et la **Chaîne de Responsabilité**.

3.1 Patron Builder (Monteur)

3.1.1 Motivation

Un document SEPA ISO 20022 comporte de nombreux objets imbriqués. Le patron **Builder** permet de décomposer l'assemblage de ce document complexe en étapes atomiques explicites, sécurisées par un contrôle d'état strict (*fail-fast*). Il masque la complexité technique d'instanciation, bannit les structures de classes internes parasites, garantit des valeurs par défaut conformes au standard bancaire et calcule automatiquement le nombre d'opérations et la somme de contrôle financière.

3.1.2 Correspondance des Acteurs

Le tableau suivant présente le rôle de chaque classe dans le patron :

Rôle GoF	Classe SEPAHub	Rôle dans le projet
Builder	<code>ISepaDocumentBuilder</code>	Interface déclarant les étapes de montage (<code>reset</code> , <code>setGroupHeader</code> , <code>beginTransaction</code> , <code>buildTransaction</code> , <code>getResult</code>).
ConcreteBuilder	<code>SepaDocumentBuilder</code>	Monteur concret GoF sans classe interne parasite, avec garde-fous d'état et calcul automatique des totaux.
Director	<code>SepaDocumentDirector</code>	Propose des constructions types (ex : document de test avec deux transactions ou document unitaire).
Product	<code>Document</code>	Objet complet représentant le document SEPA ISO 20022 (pain.008).
Client	<code>DataInitializer</code> / Tests	Utilise le monteur et le directeur pour générer des documents prêts à l'emploi.

TABLE 3 – Correspondance des acteurs du patron Builder

3.2 Patron Abstract Factory (Fabrique Abstraite)

3.2.1 Motivation

Dans la conception originale du GoF, le patron **Abstract Factory** opère en synergie naturelle avec le patron **Builder** : la fabrique est spécialisée dans l’instanciation technique des familles de sous-nœuds XML ISO 20022 (**Party**, **Account**, **Agent**, **Mandate**, **AccountSchemeId**), tandis que le monteur se focalise exclusivement sur le séquençage et la validation d’état. Cette séparation isole les détails JAXB et respecte le principe de responsabilité unique (SRP).

3.2.2 Correspondance des Acteurs

Le tableau suivant présente les acteurs du patron Fabrique Abstraite :

Rôle GoF	Classe SEPAHub	Rôle dans le projet
AbstractFactory	ISepaElementFactory	Interface déclarant les méthodes de fabrication des sous-éléments SEPA.
ConcreteFactory	SepaElementFactory	Instancie les objets JAXB avec les valeurs par défaut normatives (EUR, SEPA, RCUR).
Client	SepaDocumentBuilder	Délègue la création des briques unitaires à la fabrique injectée.

TABLE 4 – Correspondance des acteurs du patron Abstract Factory

3.3 Patron Chain of Responsibility (Chaîne de Responsabilité)

3.3.1 Motivation

Lors de la réception d'un flux SEPA, plusieurs vérifications successives sont nécessaires (schéma XSD, syntaxe XML, unicité de l'identifiant et cohérence de la somme). La **Chaîne de Responsabilité** isole chaque règle dans un maillon dédié et testable unitairement, tout en interrompant le traitement dès la première anomalie détectée (*fail-fast*).

3.3.2 Correspondance des Acteurs

Le tableau suivant présente le rôle de chaque classe dans la chaîne de validation :

Rôle GoF	Classe SEPAHub	Rôle dans le projet
Handler	<code>AbstractValidationHandler</code>	Classe abstraite gérant l'enchaînement des étapes (<code>setNext</code> , <code>checkNext</code>).
ConcreteHandler	Maillons de validation	Exécute un contrôle spécifique et passe la main au maillon suivant (<code>XsdValidationHandler</code> , <code>XmlParsingHandler</code> , <code>PmtIdUniquenessHandler</code> , <code>ControlSumValidationHandler</code>).
Context	<code>ValidationContext</code>	Objet partagé qui transporte les données le long de la chaîne.
Coordinator	<code>SepaValidationChain</code>	Service Spring qui assemble les maillons et lance la validation.
Client	<code>PostController</code>	Contrôleur REST qui appelle la chaîne lors de la réception d'un document.

TABLE 5 – Correspondance des acteurs de la Chaîne de Responsabilité

4 Description de l'architecture Docker et Déploiement

Afin d'assurer un déploiement fluide, reproductible et moderne du service SEPA26, l'architecture s'appuie sur une conteneurisation complète orchestrée par `Docker Compose`.

4.1 Architecture Multi-Services

Le projet est découpé en trois services autonomes communiquant au sein d'un réseau privé Docker (`sepa-network`) :

- **Base de données (MariaDB)** : Conteneur MariaDB 11 initialisé automatiquement avec la base `sepa26_db` et un utilisateur dédié. La persistance des transactions est garantie par un volume Docker nommé (`mariadb_data`).
- **Serveur Backend (Spring Boot 4 / Java 25)** : Conteneurisé via un `Dockerfile` multi-stage. Une première étape compile le projet avec Maven dans un conteneur dédié, puis l'artefact JAR est injecté dans une image alpine minimale avec le runtime Java 25 Temurin. Le service est lié dynamiquement à la base via les variables d'environnement (`DB_URL`, `DB_USER`, `DB_PASSWORD`).
- **Frontend (Angular)** : Application Single Page (SPA) développée en Angular, compilée puis servie par un conteneur Nginx. Nginx intègre également une configuration de reverse-proxy qui redirige les appels `/sepa26/*` vers le backend Spring Boot, éliminant tout besoin de configuration CORS dans le code Java.

4.2 Sécurité et Isolation Réseau

L'architecture Docker privilégie la sécurité :

- **Isolation réseau** : Grâce au réseau bridge `sepa-network`, la base de données MariaDB communique de manière sécurisée et privée avec le backend Spring Boot.
- **Transparence et Reverse Proxy** : Les clients accèdent au frontend sur le port 4200. Nginx achemine en interne les requêtes d'API vers le backend sur le port 8100.

4.3 Maintenance et Cycle de Vie

Grâce à `docker-compose`, la gestion du cycle de vie de la plateforme est simplifiée :

- **Démarrage complet** : `docker compose up -build -d` compile et démarre l'ensemble des conteneurs en respectant l'ordre de dépendance (la base de données démarre en premier avec vérification de son état de santé).
- **Arrêt des services** : `docker compose down` stoppe proprement les conteneurs tout en préservant le volume de données.
- **Consultation des logs** : `docker compose logs -f` permet de suivre les flux d'exécution en temps réel.

A Inventaire des Entités par Paquetage

Ce tableau exhaustif recense l'ensemble des composants logiciels, entités, services et utilitaires développés dans le backend Spring Boot, classés selon leur paquetage Java respectif :

Paquetage	Composant / Entité	Rôle et Responsabilité
fr.univrouen.sepa26	Sepa26Application	Classe principale démarrant l'application Spring Boot et initialisant le contexte applicatif.
.builder	ISepaDocumentBuilder	Interface formelle GoF déclarant les opérations abstraites de construction d'un document SEPA.
	SepaDocumentBuilder	Monteur concret GoF à étapes sans classe interne parasite, avec contrôle d'état et calcul automatique des totaux.
	ISepaElementFactory	Interface GoF Abstract Factory pour la fabrication des nœuds unitaires ISO 20022.
	SepaElementFactory	Fabrique concrète instanciant les familles de composants JAXB (comptes, agents, mandats, ICS).
	SepaDocumentDirector	Directeur GoF orchestrant la fabrication de documents SEPA préconfigurés (ex : documents types de test à deux transactions).
.config	XmlConfig	Configuration Spring déclarant sous forme de beans réutilisables le schéma XSD précompilé (Schema) et le contexte JAXB (JAXBContext).
.controllers	GetController	Contrôleur REST gérant la consultation GET (résumé XML, détail XML et détail HTML via transformation XSLT Saxon-HE).
	PostController	Contrôleur REST gérant la soumission (POST /insert) et la suppression (DELETE /delete/{id}).
	SearchController	Contrôleur REST gérant la recherche multicritères par date minimale et montant minimal (GET /search).
	IndexController	Contrôleur REST fournissant les métadonnées de santé (/) et le catalogue de documentation JSON (/help, /sepa26/help) consommé par Angular.
	RestExceptionHandler	Gestionnaire global @RestControllerAdvice normalisant les erreurs REST (JSON ProblemDetail RFC 7807 et XML SepaResponse).

Paquetage	Composant / Entité	Rôle et Responsabilité
.dto	DocumentList	DTO encapsulant une collection de documents pour le rendu XML du résumé (/resume/xml).
	EndpointDoc	Record immuable décrivant les métadonnées d'un point d'accès de l'API (URL, méthode, format, opération, description).
	SearchResponse	DTO représentant la réponse de recherche (status et DocumentList).
	SepaResponse	DTO standardisé pour les messages de retour de l'API REST (id, status, description).
.init	DataInitializer	Composant d'amorçage CommandLineRunner qui peuple la base de données au premier démarrage via le patron Builder.
.model	Document	Entité racine JPA et racine XML JAXB représentant l'enveloppe ISO 20022 (pain.008.001.02).
	CstmrDrctDbtInitn	Entité JPA et élément JAXB modélisant le bloc d'initialisation du prélèvement direct.
	GrpHdr	Entité JPA d'en-tête de lot bancaire (MsgId, CrdtTm, NbOfTxs, CtrlSum, InitgPty).
	PmtInf	Entité JPA des informations de paiement communes au lot de prélèvement (créancier, compte, date d'échéance).
	DrctDbtTxInf	Entité JPA de la transaction unitaire de prélèvement (référence PmtId, montant, mandat, débiteur).
	DrctDbtTx / MndtRltdInf	Objets intégrés (@Embeddable) encapsulant les données de prélèvement direct et de mandat (RUM, date).
	InstdAmt	Objet monétaire intégré (@Embeddable) encapsulant la valeur numérique et le code devise (Ccy="EUR").
	Objets de valeur du domaine	Classes autonomes intégrées (@Embeddable) représentant les tiers, comptes et banques (Party, Account, AccountId, PrivateId, OtherIdentification, SchemeName, Agent, FinInstnId, AccountSchemeId, PaymentTypeInfo, ServiceLevel, LocalInstrument).
	package-info	Configuration JAXB définissant l'espace de noms XML par défaut (http://univ.fr/sepa26) pour tout le paquetage.

Paquetage	Composant / Entité	Rôle et Responsabilité
.repository	DocumentRepository	Interface Spring Data JPA fournissant les requêtes CRUD, la recherche par PmtId et la recherche multicritères.
.services	IHelpService / HelpService	Service fournissant le catalogue typé de documentation des points d'accès.
	ISepaService	Interface formelle du contrat de service métier (principes DIP et OCP).
	SepaService	Service métier coordonnant la persistance, la recherche et la suppression des documents en base.
	SepaXmlService	Service dédié aux opérations XML techniques (sérialisation JAXB, validation XSD thread-safe, protection XXE).
.util	LocalDateAdapter	Adaptateur JAXB pour la conversion des dates ISO LocalDate vers xs:date .
	LocalDateTimeAdapter	Adaptateur JAXB pour la conversion des dates/heures LocalDateTime vers xs:dateTime .
	XsltTransformer	Composant utilisant Saxon-HE pour transformer les documents XML en HTML via les feuilles XSLT.
.validation	AbstractValidationHandler	Classe abstraite de base du patron Chain of Responsibility définissant le chaînage dynamique des maillons.
	ValidationContext	Contexte d'état partagé circulant à travers la chaîne (flux XML brut, document désérialisé, résultat).
	ValidationResult	Objet encapsulant le statut de validation et l'éventuel message d'erreur.
	XsdValidationHandler	Maillon validant la conformité syntaxique avec le schéma XSD officiel.
	XmlParsingHandler	Maillon désérialisant le flux XML en objet Document JAXB.
	PmtIdUniquenessHandler	Maillon métier vérifiant l'absence de doublon de PmtId en base MariaDB.
	ControlSumValidationHandler	Maillon métier vérifiant la cohérence financière de la somme de contrôle CtrlSum .
	SepaValidationChain	Coordinateur Spring assemblant la chaîne de responsabilité et orchestrant son évaluation.